

LAB INSTRUCTIONS

TO ACCESS THE LAB ENVIRONMENT

<https://linuxlab.koenig-solutions.com/>

THERE 3 LAB VMS ARE PROVIDED

ALL VM HAVE 2 USERS student AND root

PASSWORD FOR student USER IS student

PASSWORD FOR root USER IS redhat

```
=====
DOCKER
=====
```

=== DOCKER INSTALLATION ===

```
ls /etc/yum.repos.d
```

OR

```
yum repolist
```

```
yum install -y docker-ce
```

```
ip add | less
```

Note - check for the interface name docker0, it is not available

```
yum list installed | grep docker
```

```
systemctl start docker.service
```

```
ip add | less
```

Note - check for the interface name docker0, it is now available

```
systemctl enable docker.service
```

```
systemctl status docker.service
```

```
docker --help | less
```

```
docker ps --help
```

```
docker version
```

```
docker system info | less
```

```
ls /var/lib/docker
```

=== DOCKER BASIC COMMANDS ===

```
docker run quay.io/gauravkumar9130/nginxdemo
```

```
docker images -a
```

Note

- Open second tab in the same terminal window and execute "docker ps"

- press Ctrl + c in first tab

```
docker ps
```

```
docker ps -a
```

```
docker start <container-id>/<container-name>
```

```
docker ps
```

```
docker stop <container-id>/<container-name>
```

```
docker ps -a
docker start <container-id>/<container-name>
docker ps
docker pause <container-id>/<container-name>
docker ps
docker unpause <container-id>/<container-name>
docker ps -a
```

```
cd /var/lib/docker/containers/<container-id>
ls
cat hosatname
less <container-id>-.json.log
cd
```

```
docker create quay.io/gauravkumar9130/nginxdemo
docker ps
docker ps -a
```

```
docker images
cd /var/lib/docker/image/overlay2/imagedb/content/sha256/
ls
docker ps -a
docker rm <container-id>
docker ps -a
ls /var/lib/docker/containers
docker rm -f <container-id>
docker ps -a
ls /var/lib/docker/containers
```

```
docker run -d quay.io/gauravkumar9130/nginxdemo
docker ps
docker rm -f <container-id>
docker ps -a
docker images
docker rmi quay.io/gauravkumar9130/nginxdemo
docker images
ls /var/lib/docker/image/overlay2/imagedb/content/sha256/
```

```
docker pull quay.io/gauravkumar9130/nginxdemo
docker images
ls /var/lib/docker/image/overlay2/imagedb/content/sha256/
```

```
docker run quay.io/gauravkumar9130/ubuntu-git
docker images
docker ps -a
docker run quay.io/gauravkumar9130/ubuntu-git sleep 120
Note
```

- Open second tab in the same terminal window and execute "docker ps"
- Return back to first tab and execute "docker ps -a"

```
docker run -it quay.io/gauravkumar9130/ubuntu-git
pwd
exit
docker ps
```

```
docker ps -a
docker run -i quay.io/gauravkumar9130/ubuntu-git
pwd
ls /
exit
docker run -t quay.io/gauravkumar9130/ubuntu-git
pwd
ls /
Note - execute below commands in second tab
docker ps
docker stop <cont-id>
Note - Return back to first tab and see the outcome
```

```
docker ps -qa
docker rm -f $(docker ps -qa)
docker ps -a
docker run -itd --name=webapp quay.io/gauravkumar9130/ubuntu-git
docker ps
```

```
docker rename webapp webapp2
docker ps
docker exec <cont-id> hostname
docker exec webapp2 ps -ef
docker exec -it webapp2 /bin/bash
ps -ef
exit
docker ps
```

```
docker inspect webapp2 | less
docker stats
docker top webapp2
docker system events --since 60m
docker ps
docker kill webapp2
docker ps -a
docker rm webapp2
docker run -it --rm quay.io/gauravkumar9130/ubuntu-git
ls
exit
docker ps -a
```

```
docker run -itd --restart always quay.io/gauravkumar9130/ubuntu-git
systemctl restart docker.service
docker ps
```

```
docker run -itd --restart on-failure quay.io/gauravkumar9130/ubuntu-git
docker exec <container-id>/<container-name> ping
docker ps
```

=== PORT PUBLISHING / BINDING ===

Note - Open second tab [TAB2] in the same terminal window and execute the below

```
ssh root@node1
ip add | less
```

```
ping -c2<master-IP>
```

Note - Return back to first tab [TAB2] of the terminal and execute the below

```
docker run -d -p 127.0.0.1:8080:80 nginx
```

```
ip add | less
```

```
curl localhost:8080
```

TAB2

```
curl localhost:8080
```

TAB1

```
docker rm -f <container-id>
```

====

TAB1

```
docker run -d -p 8080:80 nginx
```

```
docker inspect <container-id> | grep IPAddress
```

```
curl <container-IP>:80
```

TAB2

```
curl <master-IP>:8080
```

=== DOCKER SYSTEM COMMANDS===

TAB1

```
docker system df
```

```
docker system df -v
```

=== DOCKER IMAGE LAYERS / PERSISTENCE ===

```
docker run -it quay.io/gauravkumar9130/ubuntu-git
```

```
cp /etc/passwd /tmp/mypasswd
```

```
ls /tmp
```

```
cat /tmp/mypasswd
```

```
exit
```

```
docker ps -a
```

```
docker start <cont-id>
```

```
docker ps
```

```
docker exec <cont-id> cat /tmp/mypasswd
```

```
docker rm -f <cont-id>
```

=== DOCKER CUSTOM IMAGES===

```
pwd
```

```
cd /
```

```
mkdir -v image1
```

```
cd image1/
```

```
vim Dockerfile
```

```
FROM quay.io/gauravkumar9130/ubuntu-git
```

```
RUN apt-get update
```

```
RUN apt-get -y install nginx
```

```
CMD ["nginx", "-g", "daemon off;"]
```

```
cat Dockerfile
docker build -t myimg .
docker images -a
ls /var/lib/docker/image/overlay2/imagedb/content/sha256/
docker run -d -p 80:80 --name mycontainer myimg
docker ps
curl 127.0.0.1:80
docker rm -f mycontainer
```

```
cd /
mkdir -v image2
cd image2/
gedit Dockerfile
```

```
FROM quay.io/gauravkumar9130/ubuntu-git
RUN apt-get update && apt-get -y install nginx
CMD ["nginx", "-g", "daemon off;"]
```

```
cat Dockerfile
docker build -t flatimg .
docker images -a
ls /var/lib/docker/image/overlay2/imagedb/content/sha256/
docker run -d -p 80:80 --name flat-container flatimg
docker ps
curl 127.0.0.1:80
docker rm -f flat-container
```

```
cd /
mkdir -v image3
cd image3/
gedit Dockerfile
```

```
FROM quay.io/gauravkumar9130/ubuntu-git
RUN apt-get update && apt-get -y install nginx
COPY index.nginx-debian.html /var/www/html
CMD ["nginx", "-g", "daemon off;"]
```

```
cat Dockerfile
gedit index.nginx-debian.html
WELCOME TO AKS TRAINING
```

```
docker build -t copyimg .
docker images -a
ls /var/lib/docker/image/overlay2/imagedb/content/sha256/
docker run -d -p 80:80 --name copy-container copyimg
docker ps
curl 127.0.0.1:80
docker rm -f copy-container
```

```
cd /
mkdir -v image4
cd image4/
```

gedit Dockerfile

```
FROM quay.io/gauravkumar9130/ubuntu-git
RUN apt-get update
ADD https://kplabs.in/friend.jpg/tmp
```

```
docker build -t adding .
docker images -a
ls /var/lib/docker/image/overlay2/imagedb/content/sha256/
docker run -it --name add-container adding
ls /tmp
exit
docker rm -f add-container
```

```
cd /
mkdir -v image5
cd image5/
gedit Dockerfile
```

```
FROM ubuntu:latest
WORKDIR /tmp
COPY Dockerfile .
CMD ["sleep","3600"]
```

```
docker build -t image-with-workdir .
docker container run -d --name container-01 image-with-workdir
docker container ls
docker exec -it container-01 bash
ls
pwd
exit
```

```
docker rm -f container-01
```

```
cd /
mkdir -v image6
cd image6/
gedit Dockerfile
```

```
FROM ubuntu:latest
RUN apt update && apt install iputils-ping -y
CMD ["ping", "google.com"]
```

```
docker build -t ping-image-cmd .
docker images
docker container run --name ping-container ping-image-cmd
docker container run --name ping-container-02 ping-image-cmd sleep 60
docker container ls
```

gedit Dockerfile

```
FROM ubuntu:latest
RUN apt update && apt install iputils-ping -y
ENTRYPOINT ["ping"]
```

```
docker build -t ping-entripoint .
docker images
docker container run --name ping-entripoint-01 ping-entripoint google.com
docker container ls
docker container run --name ping-entripoint-02 ping-entripoint sleep 3600
docker rm -f $(docker ps -qa)
```

=== DOCKER NETWORKING ===

```
docker network ls
docker network create -d bridge mybr
docker network ls
docker network inspect mybr | less
docker run -d -p 80:80 --name app --network=mybr quay.io/gauravkumar9130/nginxdemo
docker inspect app | grep IPAddress
```

```
docker run -it --name client1 quay.io/gauravkumar9130/ubuntu-git
apt-get install curl
curl 172.18.0.2:80 --connect-timeout 5
ip add
exit
docker run -it --name client2 --network=mybr quay.io/gauravkumar9130/ubuntu-git
ip add
apt-get install curl
curl 172.18.0.2:80 --connect-timeout 5
exit
docker ps -a
docker rm -f $(docker ps -qa)
```

=== DOCKER VOLUMES ===

```
cd /var/lib/docker/volumes
docker volume ls
docker volume create my-volume
docker volume inspect my-volume
docker run -it -v my-volume:/test --name app1 quay.io/gauravkumar9130/ubuntu-git
cd test
echo "hello"> hello.txt
ls
exit
cat /var/lib/docker/volumes/my-volume/_data/hello.txt
docker run -it -v my-volume:/test2 --name app2 quay.io/gauravkumar9130/ubuntu-git
cd test2
ls
exit
docker rm -f app1 app2
```

=====

KUBERNETES

=====

=== KUBERNETES INSTALLATION ===

ON ALL VM

```
hostname  
hostname -i
```

ON MASTER VM

```
gedit /etc/hosts
```

```
<IP-master> master  
<IP-worker1> node1  
<IP-worker2> node2
```

save and exit the /etc/hosts file

```
cat /etc/hosts  
ping -c3 node1  
ping -c3 node2  
scp /etc/hosts root@node1:/etc/hosts  
scp /etc/hosts root@node2:/etc/hosts
```

ON NODE1 VM

```
ping -c3 master
```

ON NODE2 VM

```
cat /etc/hosts  
ping -c3 master
```

ON ALL VM

```
yum install cri-o kubectl kubelet kubeadm -y  
systemctl enable --now crio.service  
systemctl status crio.service  
systemctl daemon-reload
```

ON MASTER VM

```
kubeadm init --pod-network-cidr=10.244.0.0/16 --ignore-preflight-errors=all  
mkdir -p $HOME/.kube  
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
chown $(id -u):$(id -g) $HOME/.kube/config  
ls -a  
ls -a .kube  
systemctl enable kubelet.service  
systemctl status kubelet.service
```

ON NODE1 AND NODE2 VM

```
kubeadm join <MASTER-IP>:6443 --token <RANDOM-ID> --discovery-token-ca-cert-hash sha256:<RANDOM-ID>  
NOTE - refer text editor for the above command  
systemctl enable kubelet.service  
systemctl status kubelet.service  
kubectl get nodes
```

ON MASTER VM

CALICO INSTALLATION

#wget<https://raw.githubusercontent.com/projectcalico/calico/v3.25.0/manifests/calico.yaml>

wget<https://raw.githubusercontent.com/projectcalico/calico/v3.31.5/manifests/calico.yaml>

ls

gedit calico.yaml

Note - press ctrl + h, in 'find' type docker.io, in 'replace with' type quay.io, click Replace All button and click Save button

kubectl apply -f calico.yaml

kubectl get pods -n kube-system | grep calico

kubectl get nodes

kubectl run nginx --image=quay.io/gauravkumar9130/mywebapp

kubectl get pods -o wide

kubectl delete pods --all

METAL-LB INSTALLATION

#wget<https://raw.githubusercontent.com/metallb/metallb/v0.13.7/config/manifests/metallb-native.yaml>

wget<https://raw.githubusercontent.com/metallb/metallb/v0.16.1/config/manifests/metallb-native.yaml>

ls

kubectl create -f metallb-native.yaml

kubectl get pods -n metallb-system

CONFIGURING PUBLIC IP ADDRESS POOL

gedit ip-pool.yml

apiVersion: metallb.io/v1beta1

kind: IPAddressPool

metadata:

name: first-pool

namespace: metallb-system

spec:

addresses:

- 172.25.230.10 - 172.25.230.30

kubectl create -f ip-pool.yml

=== BASIC KUBERNETES COMMANDS===

kubectl get nodes

kubectl get nodes -o wide

kubectl run nginx2 --image=quay.io/gauravkumar9130/mywebapp

kubectl get pods

kubectl get pods -o wide

kubectl get pods -A | less

kubectl exec -it nginx2 -- sh

pwd

ls

id

hostname

exit

kubectl describe pod <pod-name> | less

kubectl describe node <node-name> | less

kubectl explain pod | less

```
mkdir -v mylabs
cd mylabs
pwd
gedit multicon.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: multicon
spec:
  containers:
  - name: cont1
    image: quay.io/gauravkumar9130/mywebapp
    imagePullPolicy: IfNotPresent
  - name: cont2
    image: quay.io/opstree/redis
    imagePullPolicy: IfNotPresent
```

```
cat multicon.yml
kubectl create -f multicon.yml
kubectl get pods -o wide
kubectl exec -it multicon -- sh
exit
kubectl exec -it multicon -c cont2 -- sh
exit
kubectl get nodes -o wide
kubectl debug node/<node-name> -it --image=mcr.microsoft.com/dotnet/runtime-deps:6.0
ls
cd host
exit
kubectl get pods
kubectl delete pods --all
```

=== KUBERNETES LABELS ===

```
gedit label.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: dev-pod
labels:
  env: dev
  manager: gaurav
spec:
  containers:
  - name: abc
    image: quay.io/gauravkumar9130/nginxdemo
```

```
kubectl create -f label.yml
kubectl get pods --show-labels
kubectl run dev-pod2 --image=quay.io/gauravkumar9130/mywebapp
kubectl get pods --show-labels
```

```

kubectl label pod dev-pod2 env=dev
kubectl get pods --show-labels
kubectl label pod dev-pod2 run-
kubectl get pods --show-labels
kubectl run dev-pod3 --image=quay.io/gauravkumar9130/mywebapp -l env=prod
kubectl run dev-pod4 --image=quay.io/gauravkumar9130/mywebapp -l env=prod
kubectl get pods --show-labels
kubectl label --overwrite pod dev-pod4 env=test
kubectl get pods --show-labels
kubectl get pods --selector env=dev
kubectl get pods --selector env!=dev
kubectl get pods --selector env=dev --selector manager=gaurav --show-labels
kubectl get pods --selector 'env in (test,dev)'
kubectl get pods --selector 'env notin (test,dev)'
kubectl label --overwrite pod dev-pod4 env- color=blue
kubectl label --overwrite pod dev-pod2 dev-pod3 env- color=blue
kubectl label pods -l color=blue color=red --overwrite
kubectl get pods --show-labels
kubectl delete pod --all

```

=== KUBERNETES REPLICASET ===

gedit rs.yml

```

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: mycontainer
          image: quay.io/gauravkumar9130/nginxdemo

```

```

kubectl create -f rs.yml
kubectl get pods -o wide
kubectl get rs -o wide
kubectl describe rs rs-web | less
kubectl scale rs rs-web --replicas=3
kubectl get rs
kubectl get pods -o wide
kubectl scale rs rs-web --replicas=2
kubectl get rs
kubectl delete pods <pod-name>
kubectl get pods -o wide
kubectl delete -f rs.yml

```

=== KUBERNETES SERVICE ===

gedit ecom.yml

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: ecommerce
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ecommerce
  template:
    metadata:
      labels:
        app: ecommerce
    spec:
      containers:
        - name: abc
          image: quay.io/gauravkumar9130/mywebapp
```

```
kubectl create -f ecom.yml
kubectl get pods -o wide --show-labels
kubectl get svc
```

PART - A: SERVICE TYPE CLUSTER IP

gedit cip-ecom.yml

```
apiVersion: v1
kind: Service
metadata:
  name: cip-ecommerce
spec:
  type: ClusterIP
  ports:
    - targetPort: 80 ##container port no
      port: 5000 ##clusterip port no
  selector:
    app: ecommerce
```

```
kubectl create -f cip-ecom.yml
kubectl get svc
kubectl describe svc <svc-name>
curl <svc-clusterIP>:5000
kubectl delete -f cip-ecom.yml
kubectl get svc
```

PART - B: SERVICE TYPE NODEPORT

gedit nodep.yml

```
apiVersion: v1
kind: Service
metadata:
name: ecommerce-outside-app
spec:
type: NodePort
ports:
- targetPort: 80 ##container port no
  port: 80 ##cluster ip port
  nodePort: 30003 #####range between 30000-32767 only allowed
selector:
  app: ecommerce
```

```
kubectl create -f nodep.yml
kubectl get svc
kubectl get pods -o wide
kubectl get nodes -o wide
curl <node-IP>:30003
Note - open tab in web browser and type <node-IP>:30003
kubectl delete -f nodep.yml
kubectl get svc
```

PART - C: SERVICE TYPE LOAD BALANCE

```
gedit lb.yml
```

```
apiVersion: v1
kind: Service
metadata:
name: lb-ecommerce
spec:
type: LoadBalancer
ports:
- targetPort: 80
  port: 80
selector:
  app: ecommerce
```

```
kubectl create -f lb.yml
kubectl get svc
Note - Open a new tab in chrome browser and access the Load Balancer public IP
kubectl delete -f lb.yml
kubectl delete -f ecom.yml
kubectl get svc
kubectl get rs
```

=== KUBERNETES DEPLOYMENT ===

PART1: ROLLING UPDATE

```
vim dep.yml
```

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
name: myapp
spec:
replicas: 2
selector:
  matchLabels:
    app: myapp
template:
  metadata:
  labels:
    app: myapp
  spec:
  containers:
  - name: c1
    image: quay.io/gauravkumar9130/production:v1
```

```
kubectl create -f dep.yml
kubectl get deployment
kubectl describe deployment myapp | less
kubectl get rs
kubectl get pods -o wide
kubectl expose deployment myapp --target-port=80 --port=80 --type=LoadBalancer
kubectl get svc
kubectl set image deployment myapp c1=quay.io/gauravkumar9130/production:v2 --record
kubectl describe deployment myapp | less
kubectl rollout history deployment myapp
kubectl set image deployment myapp c1=quay.io/gauravkumar9130/production:v3 --record
kubectl rollout history deployment myapp
kubectl rollout undo deployment myapp
kubectl rollout history deployment myapp
kubectl rollout undo deployment myapp
kubectl rollout undo deployment myapp --to-revision=1
kubectl delete -f dep.yml
kubectl delete svc myapp
```

PART2: BLUE GREEN

```
vim blue.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: blue-deployment
spec:
replicas: 2
selector:
  matchLabels:
    app: nginx
  version: blue ###it can be anything
template:
  metadata:
  labels:
    app: nginx
  version: blue
```

```
spec:
containers:
- name: abc
image: quay.io/gauravkumar9130/production:v1
```

vim bgsvc.yml

```
apiVersion: v1
kind: Service
metadata:
name: bgsvc
spec:
type: LoadBalancer
ports:
- port: 80
targetPort: 80
selector:
version: blue
```

```
kubectrl create -f blue.yml
kubectrl create -f bgsvc.yml
kubectrl get svc
```

Note - Open new tab in web browser and paste the load balancer IP address

vim green.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: green-deployment
spec:
replicas: 2
selector:
matchLabels:
app: nginx
version: green ##it can be anything
template:
metadata:
labels:
app: nginx
version: green
spec:
containers:
- name: abc
image: quay.io/gauravkumar9130/production:v2
```

```
kubectrl create -f green.yml
kubectrl edit svc bgsvc
```

Note - change under selector version: green

Note - refresh the web browser tab from where you access application earlier

```
kubectrl delete -f green.yml
kubectrl delete -f blue.yml
kubectrl delete -f bgsvc.yml
```

=== KUBERNETES INGRESS ===

```
git clone https://github.com/kubernetes/ingress-nginx
cd ingress-nginx/deploy/static/provider/cloud
kubectl create -f deploy.yaml
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx
kubectl create deployment hotel --image=quay.io/gauravkumar9130/hotel --replicas=2
kubectl create deployment tea --image=quay.io/gauravkumar9130/tea --replicas=2
kubectl create deployment coffee --image=quay.io/gauravkumar9130/coffee --replicas=2
kubectl get pods
kubectl expose deployment hotel --target-port=80 --port=80
kubectl expose deployment tea --target-port=80 --port=80
kubectl expose deployment coffee --target-port=80 --port=80
kubectl get deployment
kubectl get svc
cd-
gedit ingress.yml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hotel-app
  annotations:
    kubernetes.io/ingress.class: nginx
spec:
  rules:
  - http:
      paths:
      - path: /hotel
        pathType: Prefix
        backend:
          service:
            name: hotel
            port:
              number: 80
      - path: /tea
        pathType: Prefix
        backend:
          service:
            name: tea
            port:
              number: 80
      - path: /coffee
        pathType: Prefix
        backend:
          service:
            name: coffee
            port:
              number: 80
```

```
kubectl create -f ingress.yml
kubectl get ingress
```

```
kubectl get svc -n ingress-nginx
kubectl delete -f ingress.yml
kubectl get ingress
kubectl get svc
kubectl delete svc hotel
kubectl delete svc tea
kubectl delete svc coffee
kubectl get deployment
kubectl delete deployment hotel
kubectl delete deployment tea
kubectl delete deployment coffee
kubectl get pods
cd ingress-nginx/deploy/static/provider/cloud
kubectl delete -f deploy.yaml
cd
```

=== KUBERNETES DAEMONSET ===

```
gedit ds.yml
```

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: myds
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: abc
          image: quay.io/gauravkumar9130/nginxdemo
```

```
kubectl create -f ds.yml
kubectl get ds
kubectl get pods -o wide
kubectl delete pod <pod-name>
kubectl get pods -o wide
kubectl delete -f ds.yml
```

=== KUBERNETES PERSISTENT VOLUME ===

```
kubectl get pv
kubectl get pvc
gedit pvol.yml
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv
```

```
spec:
storageClassName: hdd
accessModes:
- ReadWriteMany
capacity:
  storage: "3Gi"
hostPath:
  path: "/insidenode"
```

```
kubectl create -f pvol.yml
kubectl get pv
gedit pvclaim.yml
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: pvc
spec:
storageClassName: hdd
accessModes:
- ReadWriteMany
resources:
  requests:
    storage: "2Gi"
```

```
kubectl create -f pvclaim.yml
kubectl get pvc
gedit podpv.yml
```

```
apiVersion: v1
kind: Pod
metadata:
name: vpod
spec:
volumes:
- name: pv
  persistentVolumeClaim:
    claimName: pvc
containers:
- name: c1
  image: quay.io/gauravkumar9130/nginx
  volumeMounts:
  - name: pv
    mountPath: /usr/share/nginx/html
```

```
kubectl create -f podpv.yml
kubectl get pods -o wide
kubectl exec -it vpod -- sh
pwd
cd usr/share/nginx/html
echo "Hello, I am eks user"> test
cat test
exit
```

ON NODE-X TAB

```
cd /insidenode  
cat test
```

ON MASTER TAB

```
kubectl delete -f podpv.yml
```

ON NODE-X TAB

```
ls
```

```
kubectl delete -f pvclaim.yml  
kubectl delete -f pvol.yml
```

```
=====
```

JENKINS

```
=====
```

ON MASTER VM

```
yum list installed | grep docker  
yum install -y docker-ce  
systemctl enable --now docker.service  
systemctl status docker.service  
yum install fontconfig java-21-openjdk -y  
alternatives --config java  
java --version  
pwd  
mkdir jenkins  
cd jenkins  
git clone https://github.com/scriptcamp/kubernetes-jenkins  
ls  
cd kubernetes-jenkins  
ls  
gedit volume.yaml  
kubectl create -f namespace.yaml -f deployment.yaml -f serviceAccount.yaml -  
f service.yaml -f volume.yaml  
kubectl get pod -n devops-tools  
kubectl get svc -n devops-tools  
hostname -i
```

open browser and search master:32000

```
kubectl exec -n devops-tools -it <JENKINS_POD_NAME> -- bash  
cat /var/jenkins_home/secrets/initialAdminPassword
```

Complete initial setup

- install suggested plugins
- skip the user creation
- follow default setting
- start using jenkins

Automatically Build and Push Image to Docker Hub

- Manage Jenkins
- Plugins
- Available Plugins
- Install "Docker""Github Integration""Pipline: github""Kubernetes"

Note - after all above plugins are installed check mark restart botton, relogin further

- Manage Jenkins
- Credentials
- Add credentials
- Username with Password
- Username: root
- Password: redhat
- ID: root

Login to Docker Hub create PAT

- Click on profile
- Account Settings
- Personal Access Token
- Create new token with any name and copy token

Login to Git Hub create PAT

- Click on profile
- Settings
- On left, click developer setting
- Personal Access Token click
- Create new token with any name and copy token

- Manage Jenkins
- Credentials
- Add credentials
- Username with Password
- Username: <docker-username>
- Password: <docker-pat>
- ID: docker

- Manage Jenkins
- Credentials
- Add credentials
- Username with Password
- Username: <github-username>
- Password: <github-pat>
- ID: github

- Manage Jenkins
- Nodes
- New node
- Name: master
- Select permanent agent -> create
- Node name: master
- Remote root directory: /root
- Label: docker
- Launch method:
 - Launch agent via SSH
- Host: Master node IP
- Credentials: use root credentials
- Save

```
kubectl get pods -n devops-tools
kubectl exec -n devops-tools -it JENKINS_POD_NAME -- bash
ssh root@<IP-master>
redhat
```

create a public repository named "cicd" on github

```
cd
mkdir myimage
cd image
https://chatgpt.com/s/t\_6ac65b2cd35481919f48d2850e3ea40b
gedit Dockerfile
gedit index.html
gedit style.css
gedit script.js
ls
git init
git add .
git status
git commit -m "first commit"
git config --global user.email "user1@example.com"
git config --global user.name "user1"
git commit -m "first commit"
git remote add origin https://github.com/YOUR\_USER\_NAME/cicd.git
git branch -M main
git push origin
Note - give github username and github pat
```

Come to Jenkin page and click jenkins icon on left top

```
pipeline {
    agent {
        label 'docker'
    }

    environment {
        IMAGE_NAME = "pranavadmin/cicd"
        IMAGE_TAG = "${BUILD_NUMBER}"
    }

    stages {
        stage('Clone Code') {
            steps {
                git branch: "main",
                url: "https://github.com/pranav-admin-devops/cicd.git"
            }
        }

        stage('Verify Docker') {
```

```

steps {
sh 'docker --version'
sh 'docker ps'
}
}

stage('Build Docker Image') {
steps {
sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG .'
}
}

stage('Docker Login') {
steps {
withCredentials([
usernamePassword(
credentialsId: 'dockerhub-creds',
usernameVariable: 'DOCKER_USER',
passwordVariable: 'DOCKER_PASS'
)
]) {
sh '''
echo "$DOCKER_PASS" | docker login \
-u "$DOCKER_USER" \
--password-stdin
'''
}
}
}

stage('Push Docker Image') {
steps {
sh 'docker push $IMAGE_NAME:$IMAGE_TAG'
}
}

stage('Deploy to Kubernetes') {
steps {
sh '''
cat <<EOF > deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cicd
spec:
  replicas: 2
  selector:
  matchLabels:
    app: cicd
  template:
    metadata:
      labels:
        app: cicd
    spec:
      containers:
      - name: cicd
        image: $IMAGE_NAME:$IMAGE_TAG
        ports:
        - containerPort: 8080
'''
}
}

```

```
---
apiVersion: v1
kind: Service
metadata:
  name: cicd-service
spec:
  type: NodePort
  selector:
    app: cicd
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
EOF
```

```
echo "Applying Kubernetes resources..."
```

```
kubectl apply -f deployment.yaml
```

```
echo "Waiting for deployment..."
```

```
kubectl rollout status deployment/cicd
```

```
echo "Kubernetes resources:"
```

```
kubectl get deployment
```

```
kubectl get pods
```

```
kubectl get service
```

```
'''
```

```
}
}
```

```
stage('Cleanup') {
  steps {
    sh 'docker image prune -f'
  }
}
```

```
post {
```

```
  success {
    echo 'Docker image pushed and Kubernetes deployment completed successfully!'
  }
```

```
  failure {
    echo 'Pipeline failed'
  }
}
```